```
    jrewards = torch.stack([ex[2] for ex in exps]).detach()
5
    jmeans = torch.stack([ex[3] for ex in exps]).detach()
6
    vs = torch.stack([ex[4] for ex in exps]).detach()
7
    qs = []
    for h in range(batch_size):
8
        state = torch.cat((jobs[h].flatten(),jmeans[h]))
        qs.append(qfunc(state.detach(),param,layers=layers))
9
    qvals = torch.stack(qs)
    target = qvals.clone().detach()
    target[:,jacts] = jrewards + gamma * torch.max(vs,dim=1)[0]
10
    loss = torch.sum(torch.pow(qvals - target.detach(),2))
    losses.append(loss.detach().item())
    loss.backward()
    with torch.no_grad():
11
        param = param - lr * param.grad
    param.requires_grad = True
    return np.array(losses).mean()
```

- ***1*** Generates a random list of indices to subset the experience replay
- ***2*** Subsets the experience replay buffer to get a mini-batch of data
- ***3*** Collects all states from the mini-batch into single tensor
- ***4*** Collects all actions from the mini-batch into a single tensor
- ***5*** Collects all rewards from the mini-batch into a single tensor
- ***6*** Collects all mean field actions from the mini-batch into a single tensor
- ***7*** Collects all state values from the mini-batch into a single tensor
- ***8*** Loops through each experience in the mini-batch
- ***9*** Computes Q values for each experience in the replay
- ***10*** Computes the target Q values
- ***11*** Stochastic gradient descent

This function works pretty much the same way we did experience replay with the 2D Ising model in listing 9.12, but the state information is more complicated.